

Reasoning models changed the economics of AI. Every query now burns a variable, invisible, ungoverned compute budget before producing a single visible token. Platform teams have no control surface, no policy layer, no audit trail, and no cost attribution for this new spend. InferaCordon is the control plane that fixes this.

Part I: The Problem

1.1 How Reasoning Models Changed the Cost Surface

For three years, the economics of running language models in production were straightforward. You paid for input tokens and output tokens. Both were visible. Both were controllable via `max_tokens`. Platform teams could budget, attribute, and optimize them using standard software engineering tools. Reasoning models ended this.

When you send a request to DeepSeek-R1, QwQ-32B, or OpenAI o3, the model does not immediately produce an answer. It first generates an internal chain of thought — a private reasoning trace that can span hundreds or thousands of tokens before a single visible word reaches the client. These tokens are billed in full. They consume context window. They dominate inference latency. And they are completely invisible to every standard monitoring, cost attribution, and governance tool in the LLMOps ecosystem.

A query asking "what is 15% of 200" and a query asking "prove the correctness of this distributed consensus algorithm" both enter the same request queue. They both go through the same gateway. They both hit the same model. But the first query needs three arithmetic steps and the second needs complex multi-stage reasoning. Without a control plane that understands this difference, the platform allocates identical compute to both — wasting resources on the first and potentially truncating the second.

This is not a model problem. It is a platform problem.

1.2 The Three Production Failure Modes

Overthinking is the most common and most expensive failure mode. A reasoning model has no external signal telling it when it has thought enough. Its internal stopping heuristic is not query-aware, not budget-aware, and not policy-governed. The result is that models routinely spend 1,500-2,000 thinking tokens on queries that require 200. At production scale — tens of thousands of requests per day — this waste compounds into a significant and entirely preventable cost line.

Underthinking is the hidden quality failure. Platform teams respond to cost pressure by setting a conservative global `max_tokens` ceiling. This ceiling is applied uniformly to all requests regardless of their reasoning complexity. A multi-constraint analytical query hits the same ceiling as a simple factual lookup. The model produces a shallow, unverified answer. The quality failure is silent — no error is raised, no alert fires — because the platform has no mechanism to know that the response was produced under artificial reasoning constraint.

Budget blindness is the structural failure that enables the first two. There is no per-request reasoning budget. There is no tenant-level cost attribution for thinking tokens. There is no SLO-aware allocation that gives more compute to complex queries and less to simple ones. There is no policy engine that governs what models are allowed to spend. There is no audit trail recording what was spent and why.

Platform teams are operating AI infrastructure without the instruments they would consider basic requirements for any other compute resource.

1.3 Why Existing Tools Do Not Solve This

Every LLMOps platform in the market — LangSmith, Helicone, Portkey, Braintrust, Weights and Biases Weave — operates on the same fundamental model: observe what happened after inference completes, then present dashboards and alerts about it.

This is reactive instrumentation on top of an ungoverned process. It tells you what the model spent after the spending occurred. It does not allocate budgets before inference begins. It does not enforce policies at request time. It does not stop inference when the marginal value of additional reasoning has dropped below a threshold. It does not route requests across model tiers based on complexity. It does not attribute costs to specific tenants, teams, or features with governance-grade audit trails. The gap is not in observability. The gap is in control. Reasoning compute needs a control plane — the same class of system that CPU schedulers, memory allocators, and network rate limiters represent for other compute resources. That is what InferaCordon builds.

1.4 The Scale of the Problem

Reasoning models are not a niche deployment. DeepSeek-R1 is open-weight and deployable on-premise. OpenAI o3 and Gemini 2.5 Pro are production staples at enterprise scale. As reasoning models become the default choice for any task requiring more than surface-level response quality, the inference-time compute problem becomes the defining cost and reliability challenge of enterprise AI infrastructure.

A platform that recovers 20% of reasoning compute waste without touching output quality does not represent a modest optimization. At ten thousand requests per day at an average cost of \$0.05 per request, a 20% recovery is \$36,500 per year — from a single deployment. At enterprise scale this is a material infrastructure cost line. At the platform product level, it is a pricing advantage.

InferaCordon is built for this moment.

Part II: What InferaCordon Is

2.1 The One-Sentence Definition

InferaCordon is a governed AI inference control plane that makes reasoning compute schedulable, policy-bounded, cost-attributed, and quality-guaranteed — sitting between client applications and model serving infrastructure, making per-request decisions about budget, routing, stopping, verification, and escalation under explicit SLOs.

2.2 The Control-Plane Framing

The conceptual model that drives every architectural decision in InferaCordon is this: reasoning tokens should be treated as a managed compute resource, exactly like CPU time or memory bandwidth.

CPU schedulers allocate processor time per process based on priority and policy. Memory allocators bound usage and trigger reclamation when pressure rises. Network shapers enforce bandwidth ceilings per tenant and shed traffic under overload. None of these systems simply observe what happened after the fact and report it. They govern the resource allocation in real time, enforce policies, and degrade gracefully when capacity is constrained.

InferaCordon applies this paradigm to reasoning tokens. It allocates reasoning budgets per request based on task complexity and governing policy. It enforces per-tenant ceilings and fleet-level admission control. It detects when reasoning should stop and enforces that decision natively within the inference engine. It falls back to cheaper models when quality verification fails or when fleet

capacity is constrained. It records every decision in an immutable audit trail. And it presents all of this through a production-grade observability stack and a frontend designed for both operators and executive stakeholders.

2.3 The Four Pillars

InferaCordon is designed from the ground up to demonstrate mastery across exactly four domains. Every architectural decision maps to at least one of them. The system is incomplete if any pillar is missing.

Guardrails are enforced at both the input and output boundaries of every inference request. Input guardrails run deterministic PII redaction using Microsoft Presidio, prompt injection classification using an ONNX-exported DeBERTa model, and safety classification using an ONNX-exported Llama Guard model. Output guardrails re-run the safety classifier on every final answer before delivery, catching unsafe conclusions that emerged from benign inputs through the reasoning process. Guardrail results are recorded in every trace and audit record. Every guardrail block produces a structured refusal with an audit event reference ID.

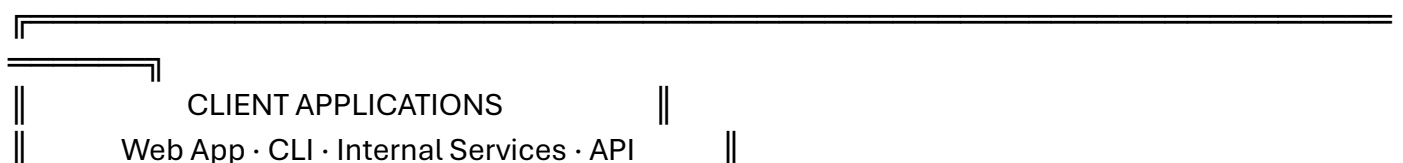
Governance is implemented as a versioned policy engine where every request is governed by an explicit, human-readable YAML policy document specifying reasoning budget limits, allowed model routes, required verification, guardrail configuration, tenant cost ceilings, latency SLOs, and data handling restrictions. Every policy version is immutable. Every decision made by the system is recorded in an append-only audit log referencing the exact policy version, model version, prompt template version, guardrail model version, and verifier version that governed it. This audit trail is the governance artifact — what a compliance team, a security team, or a legal team would inspect.

Cost Optimization is implemented at three layers. The semantic cache avoids inference entirely for requests with high similarity to recent queries. Budget class routing directs low-complexity requests to a cheap non-reasoning model and high-complexity requests to the reasoning model with appropriate token ceilings. Verification-triggered escalation ensures that the platform attempts the cheapest sufficient model first and only escalates when quality verification fails, attributing all retry costs to the original request. The primary cost metric — cost per correct answer — is computed continuously and surfaced in real time, making the economic value of the platform immediately legible.

Inference Optimization is implemented as a native vLLM LogitProcessor that enforces per-request reasoning token budgets without invasive PyTorch hooks, CUDA Graph disruption, or hidden-state extraction. When a request's reasoning token count reaches its budget ceiling, the processor injects a strong positive logit bias on the model's end-of-thinking delimiter token. The model samples the delimiter, transitions into final answer generation within the same KV-cache sequence, and continues to completion — all in a single generation stream with no second prefill cost. Token-level output entropy is logged as a telemetry signal for every request, feeding an offline analysis notebook that determines whether entropy-based adaptive early stopping warrants promotion to a production feature.

Part III: Architecture

3.1 System Architecture Diagram



HTTPS + API Key + traceparent

FASTAPI GATEWAY

Auth · Tenant ID Resolution · Rate Limiter (Token Bucket)
Request Size Validation · traceparent Injection

PRE-INFERENCE PIPELINE [50ms cap]

Complexity Scorer (Heuristic, <3ms) Policy Engine Lookup (In-memory cache, <1ms)
PII Redaction (Presidio, ~12ms) Fast-Path Circuit Breaker (fires if >50ms total)

BUDGET CONTROLLER

Complexity Score → Budget Class (low/med/high/crit)
Priority Tier Assignment (complexity × risk × sched)
Admission Control (semaphore + queue depth check)
Decision Record → Audit Log

CONTEXT ENGINE

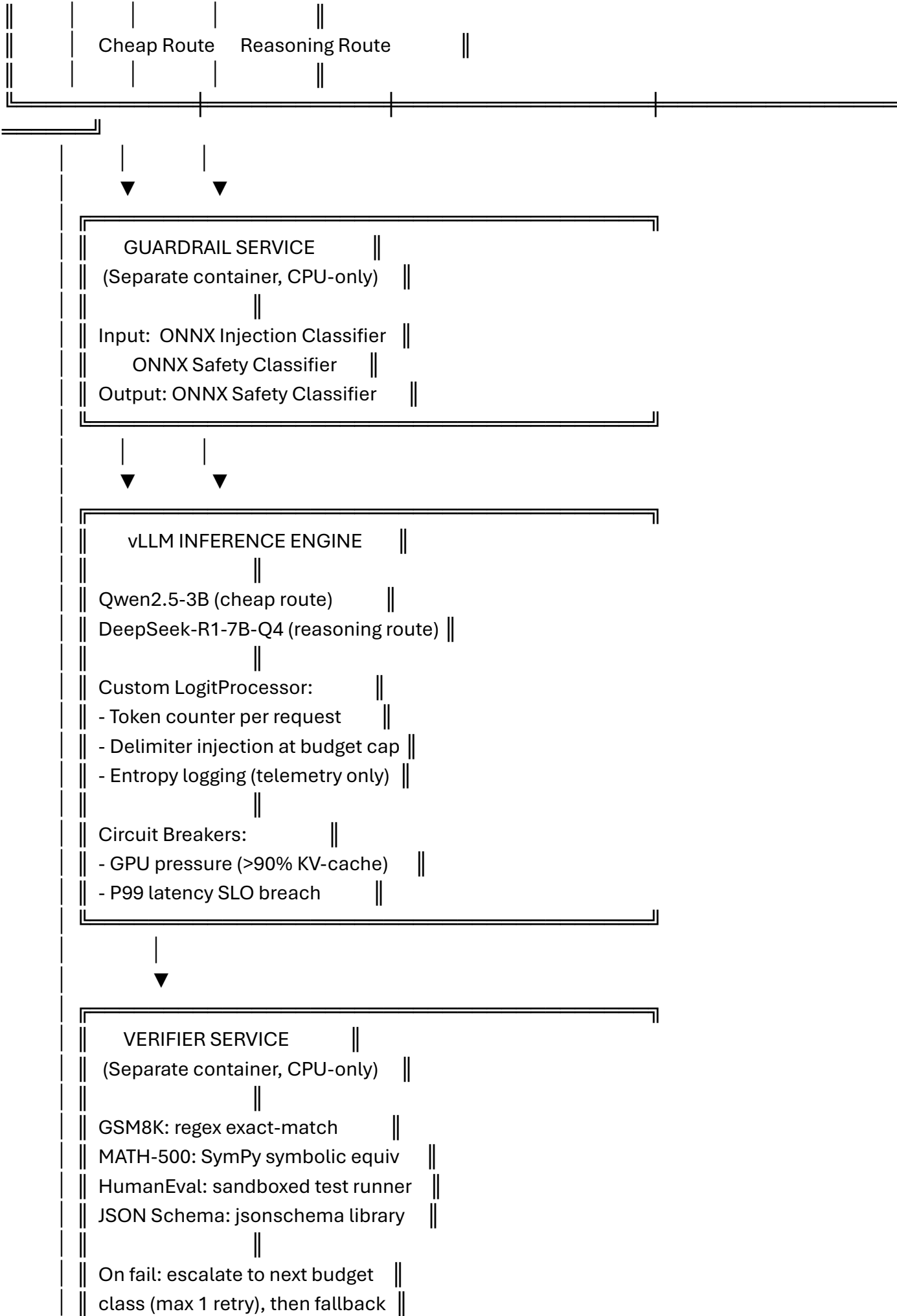
Prompt Template Selection (complexity → system prompt)
Semantic Cache Lookup (FAISS cosine similarity)

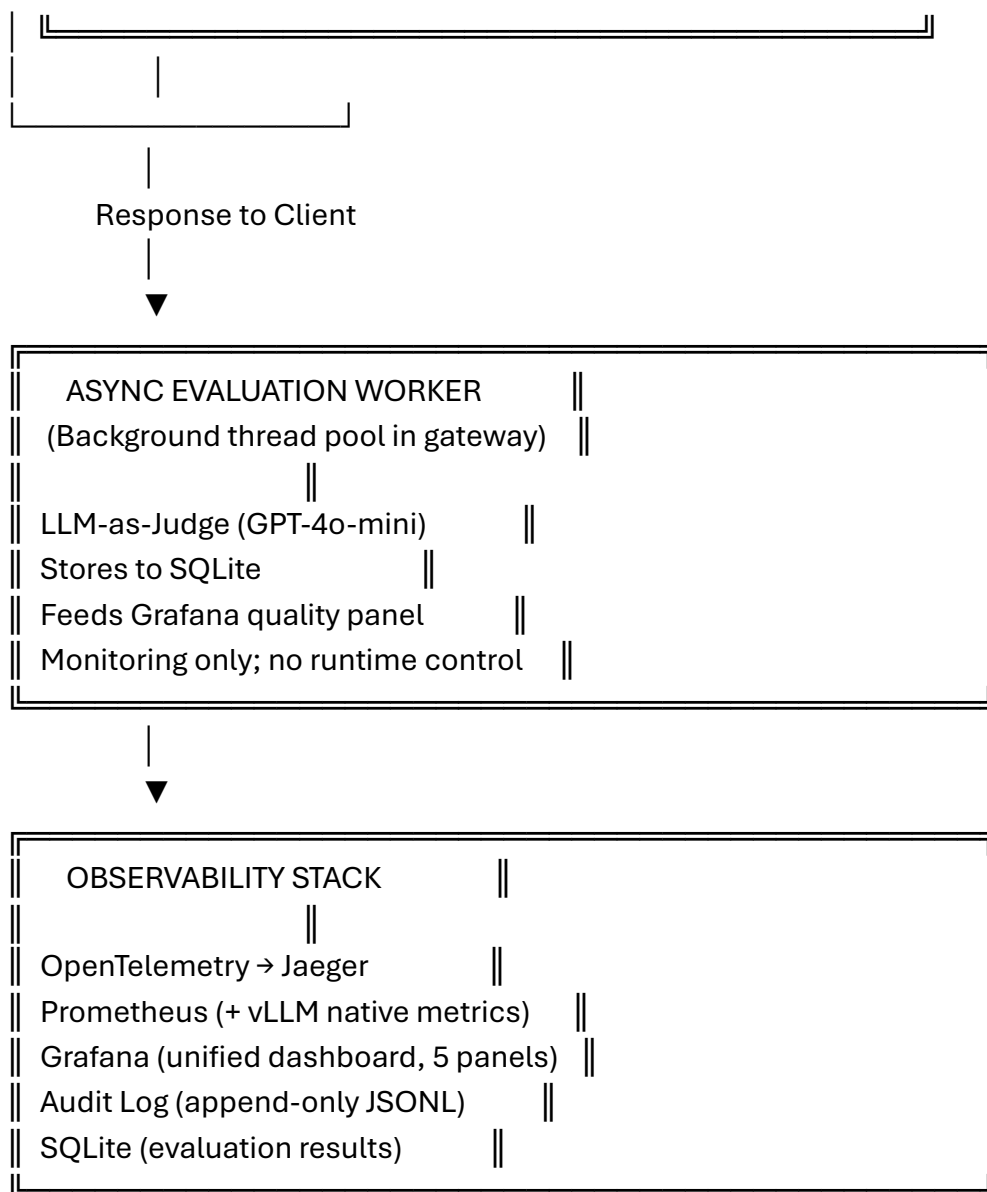
Cache Hit

Cache Miss

MODEL ROUTER

budget_class == low?
policy allows cheap?





3.2 Request Lifecycle: Three Representative Paths

Path A — Simple Query, Cache Miss, Cheap Route

A tenant submits "Summarize this paragraph in two sentences." The gateway authenticates the request, resolves the tenant ID, and checks the rate limiter. The pre-inference pipeline scores complexity as 2.1 — low class. Policy lookup returns the tenant's active policy. Presidio finds no PII. The fast-path timer shows 18ms elapsed. The budget controller assigns budget class low and priority tier standard, and writes a decision record to the audit log. The context engine prepends "Answer directly and concisely." FAISS finds no cache match with similarity above 0.92. The model router selects the cheap route. The guardrail service runs the injection classifier — score 0.02, no action. Qwen2.5-3B generates a response in 340ms. The output guardrail safety classifier returns pass. No verification is required for this budget class. The response is delivered. The async worker scores it with the LLM judge. Total cost: \$0.0004. Reasoning tokens used: 0.

Path B — Complex Query, Reasoning Route, Verification Pass

A tenant submits a multi-step algebra problem. Complexity score: 7.8 — high class. Policy assigns DeepSeek-R1-7B-Q4 with 1,024 max reasoning tokens. Context engine prepends "Think step by step. Verify each reasoning step before proceeding." Cache miss. Guardrail service runs injection classifier (0.04, pass) and safety classifier (safe, pass). The reasoning model begins generating. The LogitProcessor counts reasoning tokens. At token 743, the model reaches a natural </think> boundary

and transitions to final answer generation. Reasoning tokens used: 743 of 1,024 allocated — 281 tokens saved versus the budget ceiling, and substantially more versus natural completion which would have used approximately 1,100. The math verifier extracts the final numerical answer and confirms it matches the ground truth. Response delivered. Cost: \$0.0089. Stop reason recorded: `natural_boundary`.

Path C — Medium Query, Escalation Triggered

A tenant submits a structured data extraction task requiring JSON output matching a provided schema. Complexity score: 5.3 — medium class. DeepSeek-R1-7B-Q4 with 512 reasoning tokens. The LogitProcessor injects the delimiter at token 512 (budget exhausted). The JSON schema verifier finds the output missing a required field. Escalation triggers: the request is retried at high class with 1,024 reasoning tokens. The second response passes schema verification. Total reasoning tokens across both attempts: 1,371. Cost attributed to original request: \$0.0143, including escalation overhead. Stop reason on first attempt: `budget_exhausted`. Stop reason on second: `natural_boundary`. Escalation count in trace: 1. The frontend Playground page shows both attempts in the trace waterfall with a clear escalation span between them.

3.3 Data Flow: Audit Trail

Every request produces two persistent records in addition to the OpenTelemetry trace.

The **decision record** is written before inference begins. It captures the governance context: which policy version governed this request, which model was selected, which budget was allocated, and why. This record exists even if inference subsequently fails — it is the pre-inference governance artifact.

The **outcome record** is written after the response is delivered. It captures the execution context: tokens used versus allocated, stop reason, verification result, escalation count, actual cost, and latency breakdown. The outcome record references the decision record by request ID, creating a complete before-and-after picture for every request.

Both records are written to an append-only JSONL file that is persisted to a Docker volume. They are never overwritten, never deleted within the retention window, and never stored with raw prompt or response text by default — only derived signals, counts, and outcomes. This is the privacy-safe, compliance-oriented audit trail.

Part IV: Component Reference

4.1 FastAPI Gateway

Single entry point for all traffic. Serves the React frontend as static files on the root path. Exposes the inference API on `/v1/`. Exposes the metrics API for Prometheus scraping on `/metrics`. Exposes health check on `/health`.

The gateway is also the host for the async evaluation worker, which runs as a background thread pool within the gateway process. This eliminates the need for Celery, Redis, and a separate worker container in the MVP. The thread pool processes evaluation tasks after response delivery and writes results to SQLite without blocking the serving path.

4.2 Policy Engine

A Python library imported by the gateway — not a separate service. Policies are YAML files validated against Pydantic schemas at gateway startup. Invalid policies prevent startup rather than failing silently at runtime. Policies are loaded into an in-memory dictionary and cached for the process lifetime. A policy update requires a gateway restart.

Each policy specifies four budget profiles (low, medium, high, critical), each with a model assignment, maximum reasoning token budget, maximum output token budget, and verification requirement. Each policy also specifies guardrail configuration, tenant rate limits, cost ceilings, latency SLOs, cache settings, escalation behavior, and the version identifiers for all governed components.

The immutability guarantee is enforced by the version numbering convention: each policy file carries a monotonically increasing version integer. A loaded policy is never mutated in memory. When a new version is deployed, it is a new file with a new version number. The old version remains on disk for audit reference.

4.3 vLLM Inference Server

Runs both models on the same GPU instance using vLLM's multi-model serving capability. DeepSeek-R1-7B-Q4 on the reasoning route. Qwen2.5-3B-Instruct on the cheap route. Both served via vLLM's OpenAI-compatible HTTP API.

The custom LogitProcessor is registered per-request via vLLM's SamplingParams interface. It maintains a per-request dictionary of token counters and budget thresholds. On each generation step, it increments the counter for the active request. When the counter reaches the configured threshold, it sets the logit for the model's `</think>` delimiter token to a large positive value (typically +100.0) relative to all other logits. This makes the delimiter the near-certain next token, transitioning the model into final answer generation in the same sequence.

Per-request state is cleaned up on request completion or abort. The processor handles concurrent requests at different budget classes correctly because each request carries its own counter and threshold keyed by request ID. Concurrent access to the counter dictionary is protected by a threading lock.

vLLM's native Prometheus endpoint provides GPU utilization, KV-cache utilization, queue depth, TTFT histograms, and decode throughput metrics directly. InfaCordon does not reimplement these.

The GPU pressure circuit breaker scrapes `vllm:gpu_cache_usage_perc` every 5 seconds. Above 90% it opens and reroutes low-priority requests to the cheap model. Below 80% it closes. The latency circuit breaker monitors the P99 TTFT histogram. Above the SLO threshold it opens and applies one budget class downgrade fleet-wide.

4.4 Guardrail Service

A separate FastAPI application in a separate Docker container. CPU-only. Serves two endpoints: `/guardrail/input` and `/guardrail/output`.

ONNX models are loaded once at container startup and cached in memory. The ONNX Runtime inference session is thread-safe. The container exposes its own Prometheus metrics for latency, throughput, and error rate.

The injection classifier returns a float risk score. Scores above the threshold configured in the tenant's policy trigger an augmented system prompt. They do not block the request. Blocking injection attempts based on a classifier score with non-trivial false positive rates is a worse platform failure for enterprise workloads than the injection risk being mitigated.

The safety classifier returns a category label and confidence score. High-confidence unsafe labels block the request on both input and output paths. Medium-confidence labels are logged as audit events without blocking. The threshold for high-confidence is configurable per policy.

Guardrail service unavailability is handled by the gateway with a conservative default: block the request with a 503 and log the guardrail unavailability event. The system does not serve unguarded requests when the guardrail service is unreachable.

4.5 Verifier Service

A separate FastAPI application in a separate Docker container. CPU-only. Wraps official evaluation harnesses rather than reimplementing verification logic from scratch.

The GSM8K verifier applies a regex pattern to extract the final numerical answer and compares it to ground truth by exact numeric equality with floating-point tolerance. The MATH-500 verifier normalizes LaTeX notation and attempts exact string match, falling back to SymPy symbolic equivalence for expressions that parse as mathematical objects. The HumanEval verifier runs the standard OpenAI test harness in a sandboxed subprocess with a 2-second timeout, 256MB memory limit, and no network access. The JSON schema verifier uses Python's jsonschema library.

Verifier unavailability is handled by the gateway with a non-blocking default: return the response, mark verification as pending in the trace, and log the verifier unavailability event. The platform does not block response delivery when the verifier is unreachable.

4.6 Semantic Cache

An in-process FAISS flat index with sentence-transformer embeddings. The embedding model (all-MiniLM-L6-v2) is loaded at gateway startup and runs on CPU. On each request, the prompt is embedded and compared against the index. Cosine similarity above the policy-configured threshold (default 0.92) triggers a cache return.

Cache entries include the response text, the original request metadata, the timestamp, and the TTL. Entries expire after the TTL configured in the tenant's policy. The FAISS index is persisted to a Docker volume on a configurable interval. Cache hit rate, miss rate, and TTL expiration rate are tracked as Prometheus counters.

The semantic cache is one of the most cost-effective components in the system for workloads with repeated or near-repeated queries — common in enterprise deployments with templated prompts, FAQ-style applications, and batch processing pipelines.

4.7 Observability Stack

OpenTelemetry traces span every component from gateway ingress to response delivery. The trace schema is defined in a shared library imported by the gateway, guardrail service, and verifier service. Every span records its component name, timing, decision, and outcome. A request that fails at any component produces a complete trace up to the failure point.

Prometheus receives metrics from the gateway, the guardrail service, the verifier service, and vLLM's native endpoint. Metric names follow a consistent `inferacordon_` prefix convention for all custom metrics. vLLM's native metrics are consumed directly without renaming.

Grafana is deployed with a pre-configured unified dashboard committed to the repository. The dashboard loads automatically on container startup. No manual configuration is required. The dashboard has five tab panels: Cost, Quality, Reliability, Performance, and Governance.

Jaeger receives OpenTelemetry traces via OTLP. The Jaeger UI is accessible at port 16686 for trace inspection. The frontend links directly to Jaeger trace views from the Audit Log Explorer using the trace ID.

SQLite stores LLM-as-judge evaluation results. One database file, zero additional infrastructure. The evaluation results feed the quality panel on the Grafana dashboard and the quality metrics computed by `update_readme_metrics.py`.

Audit Log is an append-only JSONL file written by the policy engine on every decision. It is persisted to a Docker volume. It is never overwritten within the configured retention window. It is the governance artifact, separate from operational logs and observability traces.

The frontend is a React single-page application served as static files by the FastAPI gateway. It is built for two audiences: the AI platform engineer who needs deep operational visibility, and the executive or compliance stakeholder who needs governance assurance at a glance. It is designed to be screenshot-worthy and demo-ready.

5.1 Design Language

The visual design uses a carefully considered dark theme that reads as premium internal tooling rather than a generic dashboard.

Color system: Base background #0D1117 (deep charcoal-black). Surface cards #161B22 (slate). Border color #21262D (subtle). Primary accent #6366F1 (deep indigo). Secondary accent #818CF8 (softer indigo). Text primary #F0F6FC. Text secondary #8B949E. Status emerald #10B981 (healthy, pass, closed). Status amber #F59E0B (warning, degraded, escalated). Status rose #EF4444 (error, blocked, open).

Typography: Inter for all UI text (weights 400, 500, 600). JetBrains Mono for all metric values, token counts, cost figures, latency readings, version identifiers, JSON data, and trace span labels. The monospace choice for all data values creates a visual hierarchy that instantly distinguishes interface text from measured values — a deliberate choice that signals engineering precision.

Spatial system: 8px base grid. Card border-radius 8px. Card border 1px solid #21262D. Card shadow 0 1px 3px rgba(0,0,0,0.4). Focused or active element glow 0 4px 16px rgba(99,102,241,0.15). Sidebar width 240px. Content area full remaining width. All transitions 150ms ease-out.

Navigation: Left sidebar with the InferaCordon wordmark at the top, followed by five navigation items with icons and labels. A small status indicator at the bottom of the sidebar shows the health state of all backend services at a glance — green when all operational, amber when any degraded, red when any down.

5.2 Page 1: Inference Playground

The primary interactive surface and the most important page for live demos. Split horizontally: left panel at 60% for the request interface and response, right panel at 40% for the live trace.

Left panel — Request interface: A prompt textarea with a soft indigo focus ring, and a token count estimate in the bottom-right corner in JetBrains Mono at secondary text color. Below the textarea: a tenant selector dropdown and a domain selector. A settings row below those shows a manual budget class override toggle (off by default, with a tooltip explaining "let the platform decide"), a verification toggle, and an estimated cost ceiling derived from the current policy for the selected tenant.

Left panel — Stage progress: On submission, the textarea is replaced by a compact animated stage tracker showing the current pipeline stage with its elapsed time in milliseconds as each stage completes. The stages are: Complexity Scoring → Policy Lookup → Guardrail Check → Inference → Verification → Response. Completed stages show their actual measured latency in JetBrains Mono in the accent color. The currently active stage shows a pulse animation. Failed stages show in rose.

Left panel — Response area: The final answer renders in formatted markdown below the stage tracker. Below the response, a metrics strip displays seven values in a row: Budget Class used, Reasoning Tokens (used/allocated), Stop Reason, Verification Result, Escalation Count, Total Cost USD, and End-to-End Latency ms. All numeric values in JetBrains Mono. Budget class is shown as a colored pill — emerald for low, indigo for medium, amber for high, rose for critical.

Right panel — Trace waterfall: As pipeline stages complete, horizontal span bars appear on a timeline. Spans are colored by component: indigo for gateway stages, slate for policy engine, cyan for complexity scorer, emerald for successful guardrail checks, amber for flagged guardrail results, rose for circuit breaker open events, violet for inference, teal for verifier. Each span shows its label and

duration. Hovering a span reveals a tooltip with its full attribute set. An escalation event shows as a distinct gap and repeat of the inference and verifier spans, visually communicating the retry.

Right panel — Raw trace: A collapsible panel below the waterfall shows the complete JSON trace record in a syntax-highlighted code viewer.

5.3 Page 2: Observability Dashboard

Five tab panels pulling live data from the Prometheus API and the SQLite evaluation database. Charts rendered with Recharts. This page means the portfolio demo does not require Grafana to be open — the frontend itself is the operational dashboard.

Cost tab: A time-series line chart of cost per request by budget class over the last hour. A large-format metric card showing current cost per correct answer in JetBrains Mono with its delta versus the natural completion baseline in a colored badge (green for improvement, red for regression). A donut chart showing cache hit rate with the hit count and miss count labeled. A horizontal bar chart showing token savings per request broken down by budget class.

Quality tab: A stacked bar chart of verification pass, fail, and escalation rates by task class over the current evaluation run. A line chart of rolling LLM judge score average over the last 24 hours with the policy-configured quality floor shown as a dashed horizontal reference line. A table of the last 20 evaluation results with request ID, task class, budget class, verification result, judge score, and a link to the full trace.

Reliability tab: Circuit breaker status shown as three large colored state indicators (GPU Pressure CB, Latency CB, Admission Control) with the time since last state transition and the current metric value that governs each. A sparkline of P95 end-to-end latency over the last 30 minutes. A gauge showing current queue depth versus the admission control ceiling. A table of the last 10 circuit breaker state transition events.

Performance tab: GPU utilization shown as a live gauge updating every 5 seconds. KV-cache utilization shown as a second live gauge. A line chart of decode throughput in tokens per second over the last hour. A histogram of TTFT distribution with P50, P95, and P99 markers. Fast-path circuit breaker activation rate as a small metric card.

Governance tab: A table showing the active policy version for each registered tenant, with the timestamp of the last policy change and a link to the policy diff view. A chronological event log of governance events from the last hour: budget class overrides, guardrail blocks, fast-path activations, circuit breaker transitions, rate limit hits, and admission control rejections. A bar chart showing budget class distribution across today's total traffic.

5.4 Page 3: Policy Manager

A read-only view of all loaded policies. Each policy is rendered as a structured card with four collapsible sections: Budget Profiles (showing each class's model, token ceilings, and verification requirement as a compact table), Guardrail Configuration, Rate Limits and Cost Ceilings, and Version Identifiers. All version identifiers are shown in JetBrains Mono.

A version diff panel at the bottom of each policy card shows the structural changes between the current version and the previous version, rendered as a two-column diff with additions highlighted in emerald and removals highlighted in rose.

A policy validation panel at the bottom of the page accepts raw YAML input in a code editor with syntax highlighting. As the user types, the editor validates the YAML against the Pydantic schema in real time and shows either a success summary of the parsed policy or a structured error list with field names, expected types, and violation descriptions. This panel is for inspection and testing — it does not deploy policies.

This page exists because governance is only credible when humans can read, understand, and audit the policies governing the system. A policy engine whose rules are only visible in code is not a governance tool.

5.5 Page 4: Audit Log Explorer

A full-featured explorer for the audit log JSONL file. The table shows: timestamp, tenant ID, request ID, budget class (colored pill), route, stop reason, reasoning tokens used versus allocated, verification result, escalation count, and estimated cost. All timestamp and cost values in JetBrains Mono. A filter bar at the top provides dropdowns for tenant, budget class, stop reason, verification result, and escalation count, plus a date range picker. A search field filters by request ID or prompt hash prefix. Selecting a row expands an inline detail panel showing: the complete decision record JSON, the complete outcome record JSON, the linked trace ID with a click-through to the Jaeger trace view, the linked evaluation result from SQLite if available, and the policy version that governed the request with a link to the Policy Manager view for that version.

The audit log explorer is the compliance showcase. It answers the question "why did the platform make this decision for this request at this time" with a complete, versioned, auditable record. This is what separates InferaCordon from a monitoring tool and makes it a governance tool.

5.6 Page 5: System Health

A live status page for all components. Updated every 10 seconds. Five component cards: Gateway, vLLM Serving Layer, Guardrail Service, Verifier Service, and Evaluation Worker. Each card shows: a large status indicator (Operational in emerald, Degraded in amber, Down in rose), the component version, current request rate, error rate over the last 5 minutes, P95 latency, and the last three alerts from that component. The circuit breaker state for the inference layer is shown as a dedicated indicator pair (GPU Pressure, Latency) with current metric values and the timestamp of the last state transition. At the bottom of the page, a Failure Injection Panel (visible only when the DEMO_MODE=true environment variable is set) displays ten buttons, one per defined failure mode. Each button sends a test request to the gateway's /admin/inject-failure endpoint that triggers the defined failure behavior for a single request and streams the result — including the system's response — back to the UI in real time. This is the demo equivalent of a chaos engineering run. Showing the system handle failures gracefully, in real time, on a live request, is the most compelling demonstration possible.

Part VI: Failure Mode Catalog

Ten failure modes. All defined before implementation begins. All implemented with explicit handlers. All testable via the Failure Injection Panel. All with observable behavior in the frontend and in the audit log.

#	Failure	Detection Mechanism	System Response	Observable Effect
1	Complexity scorer timeout (>5ms)	Deadline timer	Fast-path: assign medium budget class, score logged as -1	Yellow "Fast Path" badge on Playground trace
2	LogitProcessor exception	Exception handler in processor	Disable adaptive stopping for request; run to hard budget cap; log error span	Amber span in trace; stop_reason = budget_exhausted

#	Failure	Detection Mechanism	System Response	Observable Effect
3	GPU utilization > 90%	Prometheus scrape every 5s	Open GPU pressure circuit breaker; reroute low-priority to cheap model	Red circuit breaker indicator on Health page
4	P99 TTFT SLO breach	Prometheus histogram	Open latency circuit breaker; reduce all new requests by one budget class	Amber latency alert on Reliability tab
5	Input guardrail service timeout	Gateway HTTP client timeout	Conservative block; 503 to client; audit event logged	Rose span in trace; guardrail timeout event in Governance tab
6	Output safety classification: unsafe	Safety classifier result	Structured refusal to client; audit event with reference ID	Rose response card in Playground; event in Audit Log
7	Policy engine cache miss (tenant unknown)	Key lookup miss	Load default policy; log audit event; serve request	Amber "Default Policy" badge in trace
8	Rate limit exhausted	Token bucket empty	429 to client with Retry-After; audit event	Error state in Playground with countdown timer
9	Trace/metrics backend unavailable	OTLP export error	Buffer telemetry locally up to configured limit; retry; never block serving	Warning indicator on Health page; no client-visible impact
10	Fallback model (cheap model) unavailable	Health check failure	503 to client with retry window; audit event; alert	Error state in Playground; alert on Health page

Part VII: Evaluation Framework

7.1 The Primary Metric

Cost per correct answer is the metric that captures the complete value proposition of InferaCordon. It is the only metric that simultaneously reflects inference cost, controller overhead, verification overhead, and answer quality. A system that reduces tokens by 30% but increases errors by 10% and adds 50ms of controller overhead may be a net regression on this metric. Only this metric reveals that.

$$C_{\text{online}} = (\text{model inference cost} \\ + \text{guardrail service cost} \\ + \text{verification cost} \\ + \text{serving overhead}) \\ / \text{number of answers meeting quality threshold}$$

$$C_{\text{total}} = C_{\text{online}} \\ + \text{async evaluation cost} \\ + \text{telemetry and storage cost}$$

Both metrics are computed and reported separately. Async evaluation cost is not charged to the online serving metric because it does not block the response path.

7.2 Baselines

Six baselines evaluated in order before any optimization component is enabled. The adaptive system must improve upon the fixed medium budget baseline to justify its existence.

Natural completion with no budget control, no routing, and no stopping. Fixed low budget (25th percentile of natural completion token usage). Fixed medium budget (50th percentile). Policy-selected budget class with no adaptive stopping and no escalation. Policy-selected budget class with escalation but no LogitProcessor stopping. Full InferaCordon with routing, LogitProcessor stopping, escalation, circuit breakers, and semantic cache.

7.3 Datasets

GSM8K: 100 examples. Exact-match numerical verification. Ground truth available. No LLM judge used.

MATH-500: 100 examples. Normalized symbolic verification with SymPy fallback. Ground truth available. No LLM judge used.

HumanEval: 50 examples. Sandboxed test runner. Pass@1 metric. No LLM judge used.

Enterprise Synthetic: 200 examples across summarization, structured JSON extraction, multi-constraint reasoning, long-context analysis, ambiguous instruction handling, unanswerable question handling, and schema compliance tasks. LLM judge used as one quality signal. 10% human-reviewed sample to calibrate judge bias.

Adversarial: 50 examples. Prompt injection attempts, PII in prompts, pathological long inputs, schema violation attempts, and requests designed to trigger each of the ten defined failure modes. Pass criterion is correct system behavior (block, log, escalate, fallback) rather than answer quality.

7.4 Ablation Study

The ablation study is the portfolio's strongest analytical artifact. It isolates the contribution of each component to the primary metric.

Layer	Metric measured
-------	-----------------

Natural completion (no control)	C_online baseline
Fixed medium budget	Δ tokens, Δ quality
Policy routing only (no stopping)	Δ cost per correct answer
Policy routing + complexity scoring	Δ cost per correct answer
Policy routing + stopping (LogitProc)	Δ cost per correct answer
Above + verification escalation	Δ cost per correct answer
Above + semantic cache	Δ cost per correct answer
Full InferaCordon	Final Δ cost per correct answer

Each layer's contribution is reported as the marginal improvement it adds over the previous layer. Components that do not improve the primary metric are documented honestly. A disciplined negative result is stronger portfolio evidence than an undifferentiated positive result.

7.5 Quality Constraint

The regression gate in the CI pipeline enforces a non-inferiority constraint. Any configuration change that causes accuracy to fall more than 1 percentage point on verifiable tasks (GSM8K, MATH-500, HumanEval) or more than 2 percentage points on the enterprise synthetic set fails the gate and blocks deployment. The gate evaluates per-slice accuracy across task categories, not only aggregate averages.

7.6 Portfolio Targets

These are measured targets, not pre-measurement claims. The README values are populated by `update_readme_metrics.py` after each benchmark run.

Cost per correct answer improvement of at least 15% versus fixed medium budget. Reasoning token reduction of 15-30% versus natural completion. Verifiable task accuracy regression no more than 1 percentage point versus natural completion. P95 latency no worse than natural completion, with a target of 10% improvement. Controller overhead 5% or less of end-to-end request latency. Policy violation rate zero in all tested scenarios. Trace completeness 99% or higher across all requests. Fallback success rate 99% or higher under fault injection across all ten failure modes.

Part VIII: Implementation Roadmap

Week 1-2: Baseline and Core Validation

Deploy DeepSeek-R1-7B-Q4 and Qwen2.5-3B-Instruct on vLLM using Docker Compose. The compose file brings up all containers in one command from day one. Implement the LogitProcessor and validate delimiter injection on 50 diverse queries across all budget classes. This is the project's only hard gate: if injection coherence rate falls below 90%, implement the hard budget cap plus finalization instruction fallback before proceeding. Run all six baselines across GSM8K, MATH-500, and HumanEval. Record token counts, latency, GPU utilization, and cost per correct answer. Deploy skeleton gateway with trace context injection and Prometheus metrics. Calibrate the complexity scorer heuristic against baseline token usage data.

Weeks 3-4: Policy Engine and Budget Controller

Build the YAML policy engine with Pydantic schema validation, in-memory caching, and the audit log writer. Implement budget class assignment and priority tier assignment. Build the token bucket rate limiter and the admission control semaphore. Implement the fast-path circuit breaker. Implement the semantic cache with FAISS and sentence-transformers. Implement static prompt rewriting. Run the baseline evaluation with policy routing enabled and document the marginal improvement.

Weeks 5-6: Guardrails and Verifier

Deploy the guardrail service container with ONNX injection classifier and ONNX safety classifier. Implement PII redaction with Presidio in the gateway pre-inference pipeline. Implement output guardrail check after inference. Deploy the verifier service container wrapping the official evaluation harnesses. Implement verification-triggered escalation in the gateway. Run the ablation evaluation through the escalation layer.

Weeks 7-8: Reliability Layer

Implement GPU pressure and latency circuit breakers with Prometheus monitoring. Implement all ten failure mode handlers. Run fault injection tests for each failure mode using a dedicated test script and verify that each produces the documented system response and the correct audit log event. Implement local telemetry buffering for trace backend unavailability. Run load tests at 10, 50, and 100 RPS with documented hardware configuration and record the capacity profile.

Weeks 9-10: Complete Observability

Complete the OpenTelemetry trace schema across all components. Deploy Jaeger and configure the OTLP exporter. Deploy the unified Grafana dashboard from the committed JSON file. Implement the async evaluation worker thread pool and LLM-as-judge scoring. Implement the SQLite evaluation result store. Implement `update_readme_metrics.py`. Run the full benchmark suite and generate the complete cost-quality-latency report.

Weeks 11-12: Frontend

Build the React application: Playground, Dashboard, Policy Manager, Audit Log Explorer, System Health, and Failure Injection Panel. Wire all API calls and Prometheus queries. Apply the complete design language. Wire the Failure Injection Panel to the backend failure injection endpoints. Run end-to-end demo scenarios and verify that every failure mode produces the correct visible behavior in the frontend.

Weeks 13-14: Portfolio Packaging

Write the technical design document. Record the five-minute demo video covering: overthinking suppression (budget class routing cutting a reasoning budget), underthinking prevention (escalation promoting a failed response to a higher budget), policy governance (policy manager showing versioned rules), circuit breaker activation (system health page showing a breaker open event under induced GPU pressure), and audit log inspection (compliance stakeholder view of request decisions). Package the repository. Finalize all architecture diagrams. Write the portfolio case study. Run `update_readme_metrics.py` for the final time with production benchmark numbers.

Part IX: Repository Structure

inferacordon/

```
|
|  └─ docker-compose.yml      ← One command: docker compose up
|  └─ .env.example
|  └─ README.md              ← Auto-updated with measured metrics
|  └─ architecture.md
|
|  └─ gateway/
|      └─ main.py
|      └─ auth.py
|      └─ rate_limiter.py
|      └─ trace_context.py
|      └─ pre_inference_pipeline.py
|      └─ complexity_scorer.py
|      └─ semantic_cache.py
|      └─ context_engine.py
|      └─ admission_control.py
|      └─ async_eval_worker.py
|
|  └─ policy_engine/
|      └─ loader.py
|      └─ validator.py
|      └─ budget_controller.py
|      └─ audit_log.py
|      └─ policies/
|          └─ acme_corp_general_qa_v1.yaml
|          └─ demo_tenant_v1.yaml
|
|  └─ vllm_adapter/
```



```
|   ├── logit_processor.py
|   ├── circuit_breakers.py
|   └── serving_config.py
|
|   ├── guardrail_service/
|   |   ├── main.py
|   |   ├── injection_classifier.py
|   |   ├── safety_classifier.py
|   |   └── models/
|   |
|   ├── verifier_service/
|   |   ├── main.py
|   |   ├── gsm8k_verifier.py
|   |   ├── math_verifier.py
|   |   ├── humaneval_runner.py
|   |   └── schema_verifier.py
|   |
|   ├── telemetry/
|   |   ├── otel_config.py
|   |   ├── prometheus_metrics.py
|   |   └── trace_schema.py
|   |
|   ├── evaluation/
|   |   ├── benchmark_harness.py
|   |   ├── ablation_runner.py
|   |   ├── regression_gate.py
|   |   ├── llm_judge.py
|   |   └── update_readme_metrics.py
|   |
|   ├── load_testing/
|   |   ├── locustfile.py
|   |   └── workload_profiles.py
|   |
|   ├── tests/
|   |   ├── unit/
|   |   ├── integration/
|   |   └── failure_injection/
|   |
|   ├── frontend/
|   |   ├── package.json
|   |   ├── src/
|   |   |   ├── App.jsx
|   |   |   └── pages/
```

```
| | | | └─ Playground.jsx
| | | | └─ Dashboard.jsx
| | | | └─ PolicyManager.jsx
| | | | └─ AuditLog.jsx
| | | | └─ SystemHealth.jsx
| | | └─ components/
| | | | └─ TraceWaterfall.jsx
| | | | └─ MetricsStrip.jsx
| | | | └─ CircuitBreakerIndicator.jsx
| | | | └─ PolicyCard.jsx
| | | | └─ FailureInjectionPanel.jsx
| | └─ lib/
| | | └─ api.js
| | | └─ prometheus.js
| └─ dist/
|
└─ dashboards/
  └─ grafana/
    └─ inferacordon_unified.json
|
└─ notebooks/
  └─ entropy_signal_analysis.ipynb
```

Part X: The README Lead Statement

The first paragraph of the InferaCordon README is generated by `update_readme_metrics.py` after each benchmark run. No number is written by hand. Every claim is a measurement.

InferaCordon reduced cost per correct answer by **[X]%** and reasoning token consumption by **[Y]%** compared to uncontrolled natural completion, while keeping verifiable-task accuracy within **[Z] percentage points** of the baseline. P95 latency **[improved / held steady]** by **[W]%** under a mixed-complexity workload on a single **[GPU model]** node. The system maintained **[>99]%** trace completeness and **[>99]%** fallback success rate across all ten defined failure modes under fault injection. The semantic cache eliminated inference entirely for **[C]%** of requests matching prior queries above the similarity threshold. All figures are produced by the benchmark harness at `evaluation/benchmark_harness.py` and updated automatically on each run.

Final Statement

InferaCordon is the answer to a problem that every organization running reasoning models at production scale has encountered but no platform tool has directly addressed: reasoning compute is expensive, invisible, ungoverned, and unattributed. It spends itself without constraint, fails silently when constrained too aggressively, and offers no handle for the platform teams responsible for reliability, cost, and compliance.

InferaCordon provides the control plane. It makes reasoning tokens schedulable by assigning per-request budgets based on task complexity and policy. It makes them governable by enforcing versioned policy documents with immutable audit trails. It makes them cost-attributed by tracking

spending to tenants, task classes, and budget decisions with a primary metric of cost per correct answer. It makes them quality-guaranteed by verifying outputs and escalating failures before they reach users. And it makes the entire system observable, reliable, and gracefully degrading through circuit breakers, admission control, defined failure modes, and a frontend console that makes the platform legible to operators and stakeholders alike.

It is built by one engineer. It runs on one GPU node. It starts with one command. And every performance claim it makes is produced by a benchmark script, not written by hand.